COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data STructures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	July 28, 2026	**DATE SUBMITTED:**	August 1, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

SEATWORK NO. 4.1 Array-Based vs. STL Stack Implementations

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (`std::stack`).

ILO_A > main.cpp > ...

```cpp
#include <iostream>
#include <stack> // call the stack from the STL
#include <string>

struct Item {
    std::string name;
    int quantity;

    // Constructor
    Item(std::string n, int q) : name(n), quantity(q) {
        std::cout << "Item constructed: " << name << "\n";
    }
};


int main(){

        std::stack<int> newStack; // Created an Object
        std::stack<int> nextStack; // Created an Object
        std::stack<Item> inventory; // Created an Object

        // Testing the empty() function to check if the stack is empty
        std::cout << "Is the Stack Empty?? " << newStack.empty() << std::endl;

std::cout << "====================================" << std::endl;

        // Insert 3 elements and Checks & Returns the Top
        newStack.push(3);
        std::cout << "Top of the Stack: " << newStack.top() << std::endl;
        newStack.push(2);
        std::cout << "Top of the Stack: " << newStack.top() << std::endl;
        newStack.push(1);
        std::cout << "Top of the Stack: " << newStack.top() << std::endl;

std::cout << "====================================" << std::endl;

        std::cout << "Is the Stack Empty?? " << newStack.empty() << std::endl;

std::cout << "====================================" << std::endl;

        // Inserts more elements
        nextStack.push(6);
        nextStack.push(7);
        nextStack.push(8);

```

```
46          // Checks the size of the stack
47          std::cout << "Size of the Stack is: " << newStack.size() << std::endl;
48
49  std::cout << "=====================================" << std::endl;
50
51          // Swap -> Swaps the contents of the public member functions
52          newStack.swap(nextStack);
53          // Checks if the swap works
54          std::cout << "New Stack's TOP is now: " << newStack.top() << std::endl;
55
56  std::cout << "=====================================" << std::endl;
57
58          // Emplace -> Constructs and Insert Elements of public member function
59          inventory.emplace("Makeup", 7);
60          // Checks the top object
61          std::cout << "Top item in the stack rn: " << inventory.top().name << std::endl;
62
63  std::cout << "=====================================" << std::endl;
64
65      return 0;
66  }
67
```

```
PS C:\Users\TIPQC\Documents\CPE21S3_SANTIAGO\HOA4p1>
Is the Stack Empty?? 1
========================================
Top of the Stack: 3
Top of the Stack: 2
Top of the Stack: 1
========================================
Is the Stack Empty?? 0
========================================
Size of the Stack is: 3
========================================
New Stack's TOP is now: 8
========================================
Item constructed: Makeup
Top item in the stack rn: Makeup
========================================
```

Code Analysis:

The Intended Learning outcomes A taught me the basics of implementing a stack in C++ with the provided Standard template Library (STL). Through continuous implementation and checking through printing, us students are able to grasp more onto the concept of the stack implementations. The code begins by creating three separate stack objects: two for integers (newStack and nextStack) and one for custom Item structures (inventory) (Note: this is modified for the emplace function). It uses the empty() function to verify whether the stack contains elements, initially returning 1 (true) before any items are added. Elements are then added using push(), and the top() function is called to view the most recently added item, illustrating how a stack operates on a Last-In,

First-Out (LIFO) basis. The code checks the number of elements using size() and demonstrates swapping the entire contents of two stacks using swap(). Finally, it uses emplace() to directly construct and insert a new object into the inventory stack without making extra copies, displaying the name of the top item to confirm the addition.

ILO B.1: Implement a stack using a Static Array.

ILO_B > main.cpp > ...

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1;
int newData, elements, choice;

// Prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
// Displays all
void displayAll();

int main(){

    std::cout << "================================================================" << std::endl;

    std::cout << "Enter the Number of Maximum elements for the New Stack (max:100): " << std::endl;
    std::cin >> elements;



    while(true){
        std::cout << "================================================================" << std::endl; // for organization purposes
        std::cout << "Stack Operations: " << std::endl;
        std::cout << "1. PUSH \n2. POP \n3. Top \n4. isEmpty \n5. isFull \n6. Display All Elements" << std::endl;
        std::cout << "Enter Choice: " << std::endl;
        std::cin >> choice;
        std::cout << "================================================================" << std::endl; // for organization purposes

            switch(choice){
                case 1: push();
                break;
                case 2: pop();
                break;
                case 3: Top();
                break;
                case 4: std::cout << isEmpty() << std::endl;
                break;
                case 5: std::cout << isFull() << std::endl;
                break;
                case 6: displayAll();
                break;
                default: std::cout << "Invalid Choice" << std::endl;
                break;

            }
        }

    return 0;
}
```

```cpp

bool isEmpty(){
    // top is -1
    if(top == -1){
        std::cout << "Stack Underflow" << std::endl;
        return true;
    }
    return false;
}

bool isFull(){
    // not equal to the maxCap
    if(top >= elements){
        std::cout << "Stack Overflow" << std::endl;
        return true;
    }
    return false;
}

void push(){

    // Check if the Stack is Full, if yes return an error
    if(isFull()) return;
    // Ask the User for a value to be pushed into the stack
    std::cout << "What number you want to add to the stack? " << std::endl;
    std::cin >> newData;
    // Store inside the Stack
    stack[++top]= newData;
}

void pop(){
    // Check if there are no Elements inside the stack
    if(isEmpty()) return;
    // Display the Element to be popped
    std::cout << "The Element to be Popped: " << stack[top] << std::endl;
    // Decrement the top
    stack[top--] = newData;
}

void Top(){
    // Check if the stack is empty
    if(isEmpty()) return;
    // Display the Top Element for Checking
    std::cout << "The Top Element: " << stack[top] << std::endl;
}
```

```
100
101  void displayAll() {
102      // Check if the stack is empty
103      if(isEmpty()) return;
104      // Loops backwards from top down to its last element
105      for (int i = top; i >= 0; i--) {
106          std::cout << stack[i] << " ";
107      }
108      std::cout << "\n";
109  }
```

```
==========
Enter the Number of Maximum elements for the New Stack (max:100):
4
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
1
==========
What number you want to add to the stack?
78
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
1
==========
What number you want to add to the stack?
89
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
1
==========
What number you want to add to the stack?
56
==========
```

```
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
1
==========
What number you want to add to the stack?
23
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
1
==========
What number you want to add to the stack?
41
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
1
==========
Stack Overflow
==========
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
2
```

```
Enter Choice:
2
================================================================
The Element to be Popped: 41
================================================================
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
3
================================================================
The Top Element: 23
================================================================
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
4
================================================================
0
================================================================
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
5
================================================================
0
================================================================
```

```
Stack Operations:
1. PUSH
2. POP
3. Top
4. isEmpty
5. isFull
6. Display All Elements
Enter Choice:
6
================================================================
23 56 89 78
================================================================
```

Code Analysis:

The program in Intended learning outcomes B implements a stack manually, using a fixed-capacity static array of 100 elements and a menu-like interface for an interactive program for the user. It uses a global array stack[] and a top index variable initialized to -1 to keep track of the top item in the stack. The program first displays a demand to the user to set a custom element limit within the maximum capacity, then enters an interactive menu loop for better control of the program/system. Before adding or removing items, the program checks the isEmpty and isFull conditions: isEmpty() ensures top is not -1 to prevent stack underflow, while isFull() checks if top has reached the user's defined limit to prevent stack overflow. When pushing a new value, top is incremented before storing the data in the array as seen in the function definition in the code. When popping an element, the top value is displayed first for checking and the top is decremented to effectively remove it from the stack, although the memory of that popped element stays. The Top() function inspects the current top element without modifying it, and displayAll() prints all present elements by iterating backwards from top down to index 0, to list it with LIFO basis.

ILO B.2: Stacks using Linked Lists

Main.cpp

ILO_B2 > mainB2.cpp > ...

```cpp
#include<iostream>
#include "listStack.h"

int main(){
    std::cout << "---------- Pushing 'I' 'E' 'E' 'E' ---------- \n";

    // Pushing
    push('I');
    push('E');
    push('E');
    push('E');

    // Display Top
    Top<char>();
    std::cout << "----------------------------------------- \n";

    // Display All Elements in the List
    std::cout << "Display of All the ELements: ";
    displayAll<char>();

    // Pop And Print Each Element
    std::cout << "---------- Popping 'I' 'E' 'E' 'E' ---------- \n";
    while(!isEmpty<char>()){
        std::cout << "Popping: " << pop<char>() << std::endl;
    }

    // Test Stack Underflow
    std::cout << "----------------------------------------- \n";
    pop<char>();

}
```

Analysis

Main.cpp checks if the code in the header file pushes through or works. It does printing and displaying of the stack/list we currently have. In this activity it first prints headers for organization purposes then proceeds to push 4 characters (I, E, E, E). After pushing we then proceed to display the top element and all the elements present in the list. Then, we print another header for popping the elements in the list that prints the element to be popped. Lastly, it tests the current state of the stack by calling another pop function.

Header FIle

ILO_B2 > C listStack.h > push<T>(T)

```cpp
#ifndef LISTSTACK_H
#define LISTSTACK_H
#include<iostream>

// Class node with data and next members (public)
template <typename T>
class Node{
    public:
    T data;
    Node<T>* next = nullptr;
};

// Create a Pointer/Reference tip that
// always points to the current top node
// Global
template <typename T>
Node<T>* head = nullptr;

```

Analysis

This snippet creates the template. It uses a class for Node to store T data and initialize that Node next is pointed to a nullptr. Outside the public class, we then initialized a global Node head that is also pointed to nullptr. This creates a pointer or a reference top that always points to the current Top/Head of the list.

```
20  // Pushing Operations:
21  template <typename T>
22  void push(T newData){
23      // Create a newNode
24      Node<T>* newNode = new Node<T>;
25      // Assign the data to the newNode
26      newNode->data = newData;
27      // Point the newNode next to the current Top
28      newNode->next = head<T>;
29      // Assign the new Top to the newNode
30      head<T> = newNode;
31  }
```

This snippet shows a function for pushing. It first creates a new Node, then assigns the data to a new Node. After such, we then point the new Node-> to the current top. Lastly, we assign the new top to the new Node. This way we can track that the element we pushed is inserted to the stack and is also following the LIFO basis of stack (Last in, First out).

```
33  // IsEmpty for Checking:
34  template <typename T>
35  bool isEmpty(){
36      return head<T> == nullptr;
37  }
```

This snippet shows a simple function made to check whether the active stack/list is empty or not. This was made so it can be reused in other functions like pop.

```
39  // Popping Operations:
40  template <typename T>
41  T pop(){
42      // If the stack is empty error catch:
43      if(isEmpty<T>()){
44          std::cout << "Stack Underflow" << std::endl;
45      }
46      // Saving a Current Top Node
47      Node<T>* tempNode = head<T>;
48      // Store the data of the top node to a temporary variable
49      T tempVal = tempNode->data;
50      // Move the head to the next node
51      head<T> = head<T>->next;
52      // Free up the memory
53      delete tempNode;
54      // Return the temporary variable data
55      return tempVal;
56  }
```

This snippet shows the pop operation function. It first checks whether the stack/list is empty. After confirming that condition, we can then proceed to other steps. It will then save the current top node to a temporary variable. We then create a temporary Value data storer to be able to point the data of the temporary node somewhere it can be stored. After that, we can now move the head to the head-> and delete the tempNode and tempVal to erase the data remains of the previous head/node.

```
58  // Top Checking Operations:
59  template <typename T>
60  void Top(){
61      // What if ???? the stack is Empty
62      if(isEmpty<T>()) return;
63
64      std::cout << "The Top is: " << head<T>->data << std::endl;
```

This snippet shows a simple function that checks whether the stack/list is empty before it proceeds to print the Top of the active stack/list.

```cpp
template <typename T>
void displayAll(){
    // Checks if the Stack/Linked List is empty
    if (head<T> == nullptr){
        std::cout << "List is Empty" << std::endl;
        return;
    }
    const Node<T>* currNode = head<T>;
    // Prints data until reaches null
    while(currNode != nullptr){
        std::cout << currNode->data;
        // creates an arrow everytime it confirms that data is != to null
        if (currNode->next != nullptr){
            std::cout << "->";
        }
        // Considers every next data as the head
        currNode = currNode->next;
    }
    std::cout << std::endl;
}
```

This snippet shows the displayAll function. It uses the transversal method from singly linked list as reference for it to be able to go through each element in the active stack/list. This way it will print each of the elements until it reaches null.

Output	Analysis

```
---------- Pushing 'I' 'E' 'E' 'E' ----------
The Top is: E
Display of All the ELements: E->E->E->I
---------- Popping 'I' 'E' 'E' 'E' ----------
Popping: E
Popping: E
Popping: E
Popping: I
-------------------------------------------
Stack Underflow
```

This is the output of the printing and calling done in the main.cpp

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack

Header File

ILO_C > C balancedStack.h > ...

```
#ifndef BALANCEDSYMBOLS_H
#define BALANCEDSYMBOLS_H

#include <iostream>
#include <string>
using namespace std;

// Node for linked list stack
struct Node {
    char data;
    Node* next;
};

// Global top pointer
Node* topNode = nullptr;

// Push operation
void push(char c) {
    Node* newNode = new Node();
    newNode->data = c;
    newNode->next = topNode;
    topNode = newNode;
}

// Pop operation
char pop() {
    if (topNode == nullptr) {
        return '\0'; // empty stack
    }
    char val = topNode->data;
    Node* temp = topNode;
    topNode = topNode->next;
    delete temp;
    return val;
}

// Check if stack is empty
bool isEmpty() {
    return topNode == nullptr;
}

// Clear stack (for next run)
void clearStack() {
    while (!isEmpty()) {
        pop();
    }
}
```

```
48
49  // Function to check balanced symbols with detailed error reporting
50  bool checkBalanced(const std::string& expr) {
51      clearStack(); // reset before checking
52      for (char c : expr) {
53          if (c == '(' || c == '{' || c == '[') {
54              push(c);
55          } else if (c == ')' || c == '}' || c == ']') {
56              if (isEmpty()) {
57                  std::cout << "Error: Extra closing symbol '" << c << "' found." << std::endl;
58                  return false;
59              }
60              char top = pop();
61              if ((c == ')' && top != '(') ||
62                  (c == '}' && top != '{') ||
63                  (c == ']' && top != '[')) {
64                  std::cout << "Error: Expected matching for '" << top
65                            << "' but found '" << c << "'." << std::endl;
66                  return false;
67              }
68          }
69      }
70      if (!isEmpty()) {
71          std::cout << "Error: Missing closing symbol(s) for: ";
72          while (!isEmpty()) {
73              char leftover = pop();
74              std::cout << "'" << leftover << "' ";
75          }
76          std::cout << std::endl;
77          return false;
78      }
79      return true;
80  }
81
82  #endif
```

Main File

ILO_C > mainC.cpp > ...

```cpp
#include "balancedStack.h"
#include <iostream>
#include <string>

int main() {
    std::string expr;
    char choice;

    do {
        std::cout << "==================================================" << std::endl;
        std::cout << "Enter an expression: ";
        std::cin >> expr;

        if (checkBalanced(expr)) {
            std::cout << "Expression is balanced!" << std::endl;
        } else {
            std::cout << "Expression is NOT balanced!" << std::endl;
        }

        std::cout << "==================================================" << std::endl;
        std::cout << "Do you want to check another expression? (y/n): ";
        std::cin >> choice;

    } while (choice == 'y' || choice == 'Y');

    std::cout << "Program terminated. Goodbye!" << std::endl;
    return 0;
}

```

Output

```
PS C:\Users\Dred\Documents\Data Struct and Algo\HOAs\HOA4p1> cd
_C\" ; if ($?) { g++ mainC.cpp -o mainC } ; if ($?) { .\mainC }
==================================================
Enter an expression: (A+B)+(C-D)
Expression is balanced!
==================================================
Do you want to check another expression? (y/n): y
==================================================
Enter an expression: ((A+B)+(C-D)
Error: Missing closing symbol(s) for: '('
Expression is NOT balanced!
==================================================
Do you want to check another expression? (y/n): y
==================================================
Enter an expression: ((A+B)+[C-D])
Expression is balanced!
==================================================
Do you want to check another expression? (y/n): y
==================================================
Enter an expression: ((A+B]+[C-D]}
Error: Expected matching for '(' but found ']'.
Expression is NOT balanced!
==================================================
Do you want to check another expression? (y/n): n
Program terminated. Goodbye!
```

Code Analysis

This code uses a stack built with linked lists to check if symbols in an expression are balanced. It pushes opening symbols like (, {, and [onto the stack, and pops them when closing symbols appear to ensure they match correctly. If a mismatch happens or a closing symbol has no partner, the program reports an error (an added feature I wanted to add). At the end, if there are still opening symbols left, it points out which ones are missing their closing pairs. My personal modification makes the program more helpful by showing detailed error messages, such as identifying extra closing symbols or telling the user exactly which opening symbol still needs a match. I believe that this modification makes debugging easier and gives clearer feedback compared to just saying “not balanced” or “balanced”. Overall, the program was like the ILO B too but with few tweaks and modifications through research.

Learnings

In this lab activity, I learned how stacks can be built and used in different ways to solve problems. Using the STL stack showed me how easy it is to apply built-in functions, while the array application taught me how to manage memory limits and check for overflow or underflow. The linked list stack gave me more flexibility since it does not need a fixed size. For ILO C/Supplementary, I applied these ideas to check if symbols in an expression are balanced. I also added my own modification so the program can point out missing or mismatched symbols, which makes errors easier to understand. This change gave clearer feedback compared to just saying “balanced” or “not balanced.” Overall, the activity helped me understand the LIFO rule better and showed how stacks can be used in real problems like debugging.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.